Metamorphic Testing
Part IV Projects

First meeting

6 March 2026

Testing

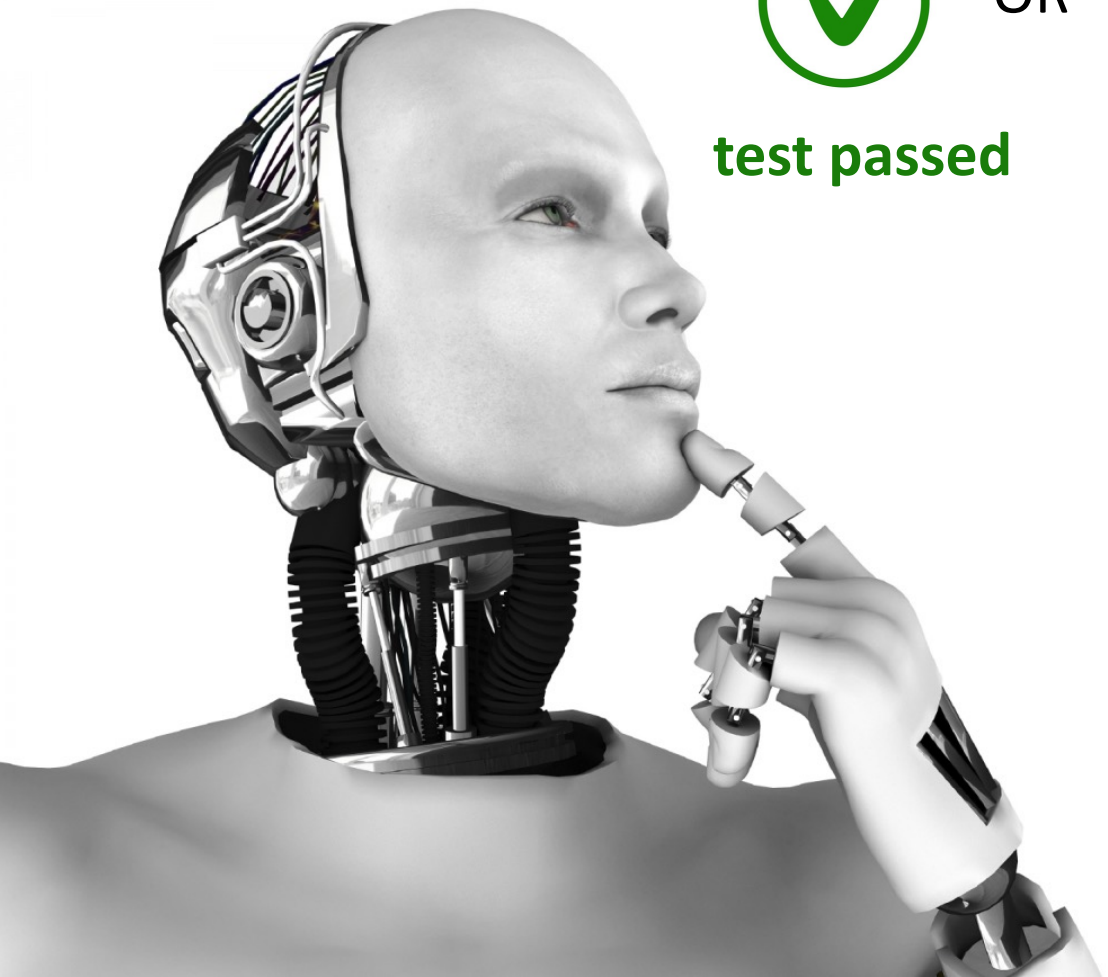
```
@Test
public void test1() {

    int max = Math.max(10, 5);
    assertEquals(10, max);

}
```

INPUT
ORACLE

Test Oracle Problem





OR

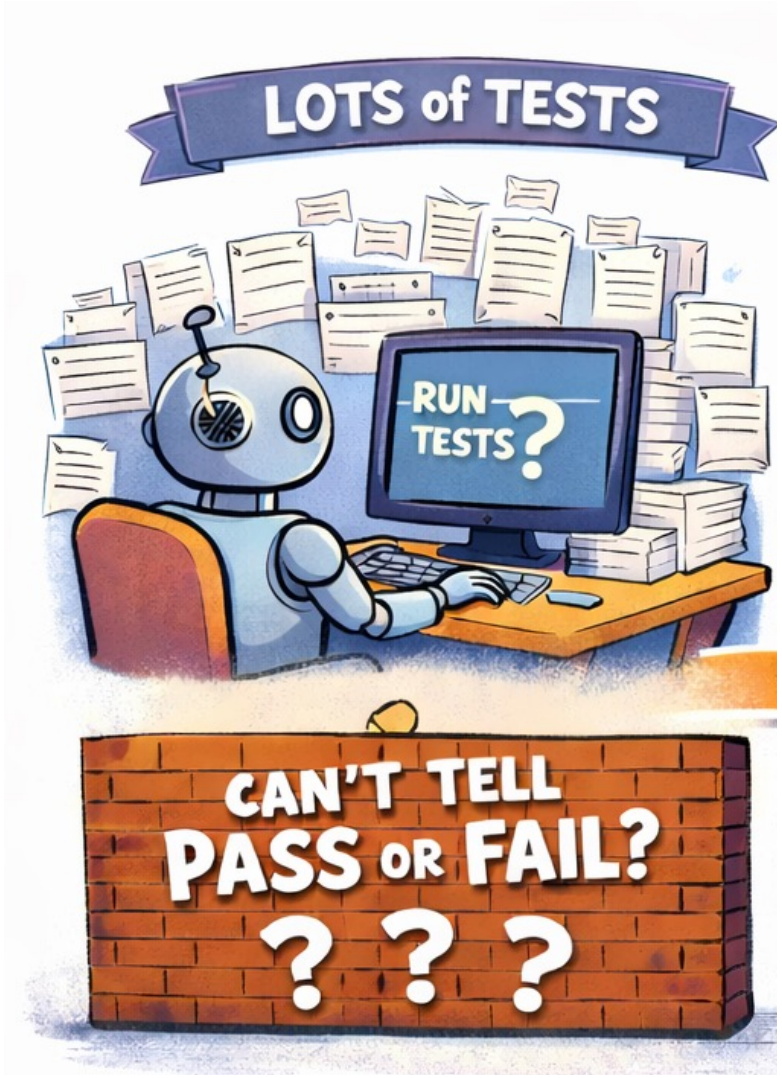
test passed **test failed**

```
@Test
public void test1() {

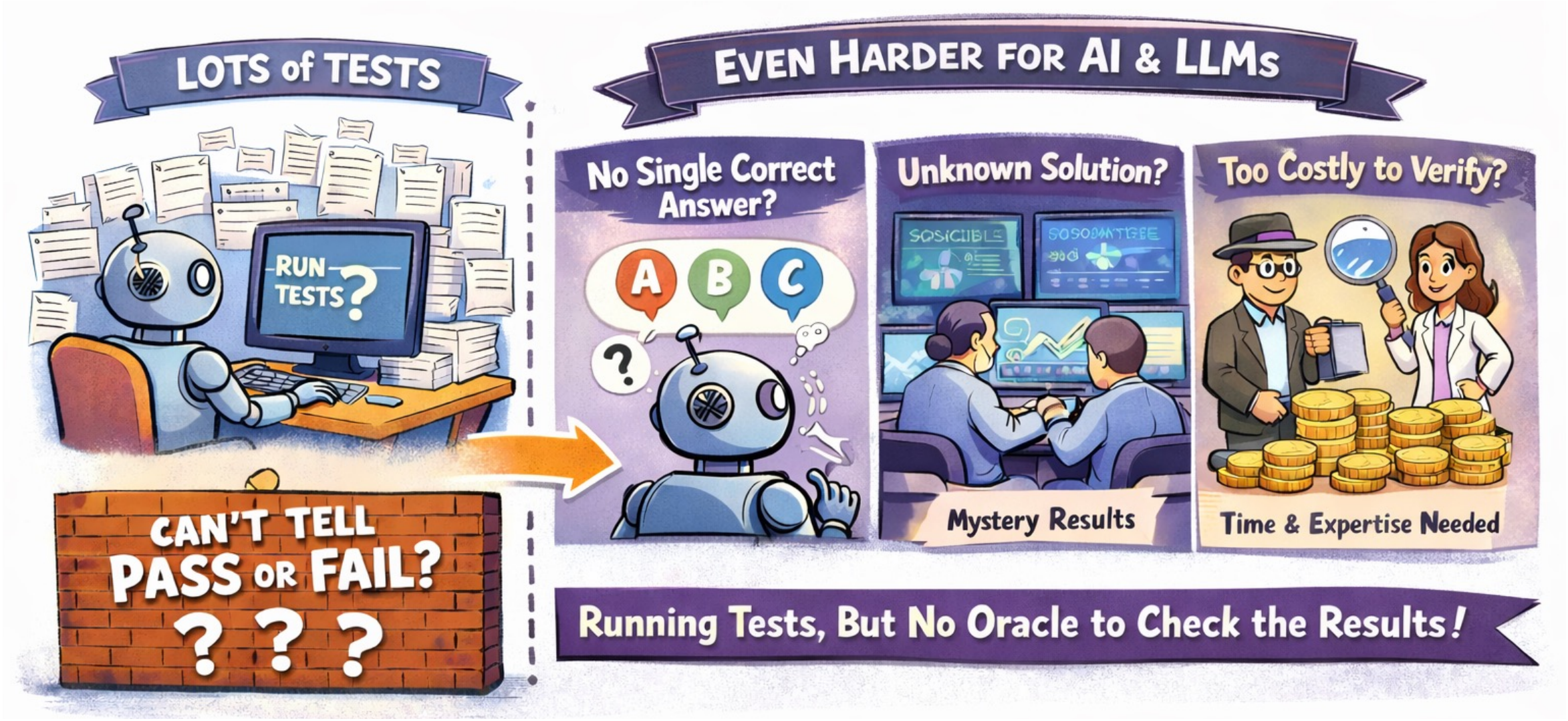
    int max = Math.max(10, 5)
    assertEquals(?????, max);

}
```

The Oracle Problem in Software Testing



The Oracle Problem in Software Testing



Metamorphic Testing





T.Y. Chen S.C. Cheung S.M. Yiu

"Metamorphic testing: A new approach for generating next test cases"
Technical Report HKUST-CS98-01 **1998**

香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Metamorphic Testing




My PhD Advisor





T.Y. Chen S.C. Cheung S.M. Yiu

"Metamorphic testing: A new approach for generating next test cases"
Technical Report HKUST-CS98-01 **1998**

香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Metamorphic Testing





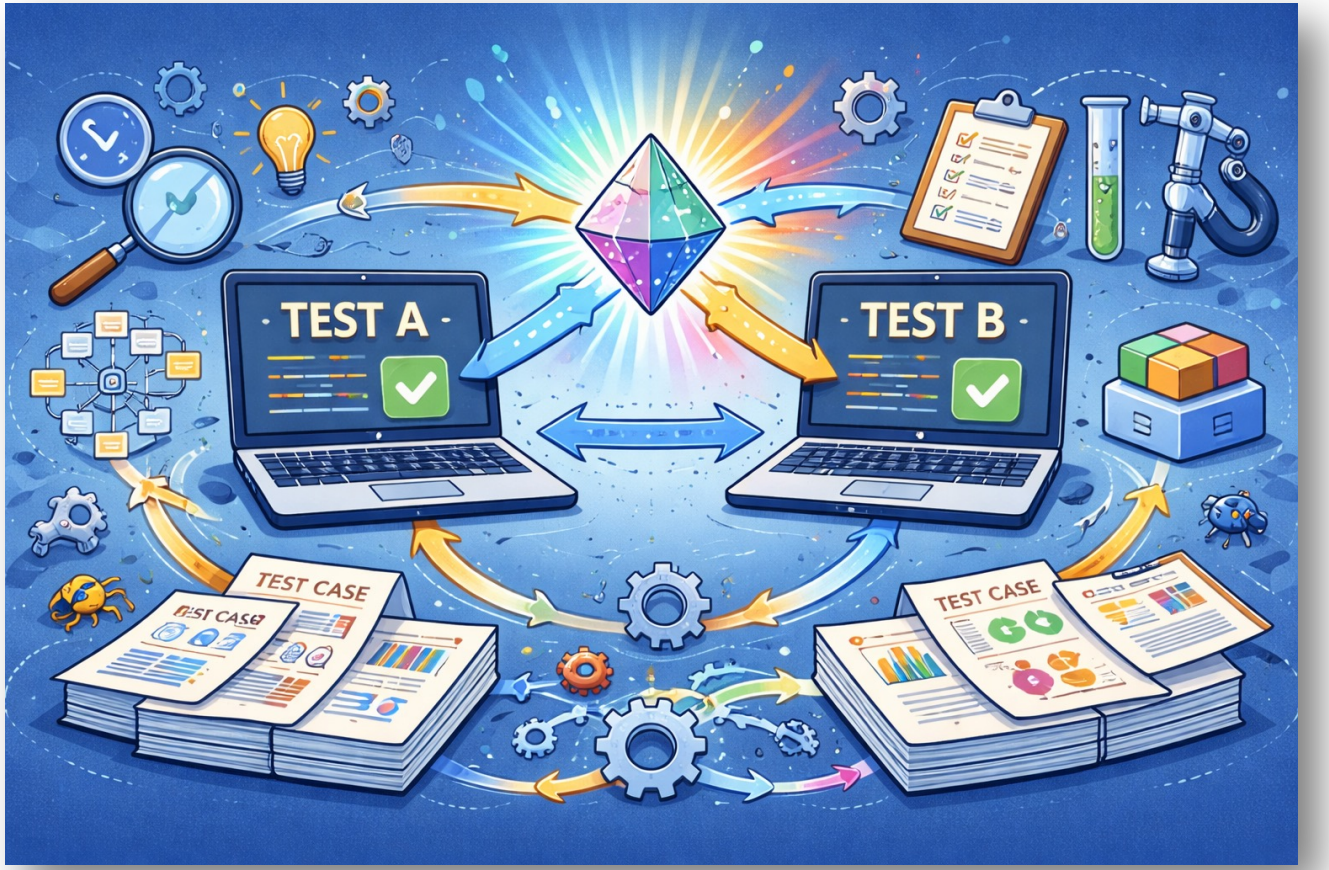
T.Y. Chen S.C. Cheung S.M. Yiu

"Metamorphic testing: A new approach for generating next test cases"
Technical Report HKUST-CS98-01 **1998**

香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Metamorphic Testing

Metamorphic testing validates a system by executing it on **multiple related inputs** and checking whether the resulting **outputs satisfy an expected relationship**, even when the exact correct output for any single input is unknown.



Metamorphic Testing (example)



Square Function: $f(x) = x^2$

```
@Test
public void test1() {
    int result= square(5)
    assertEquals(25, result);
}
```

```
@Test
public void test2() {
    int result= square(4)
    assertEquals(16, result);
}
```

```
@Test
public void test3() {
    int result= square(-4)
    assertEquals(16, result);
}
```

Metamorphic relation

necessary properties of the target function or algorithm in relation to multiple inputs and their corresponding outputs.

$$x_1 = -x_2 \rightarrow square(x_1) = square(x_2)$$

Metamorphic Relation (Test Oracle)

logical implication

pre condition → post condition

input relation → output relation

$x_1 = -x_2$ → square(x_1) = square(x_2)

p	q	implies $p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T



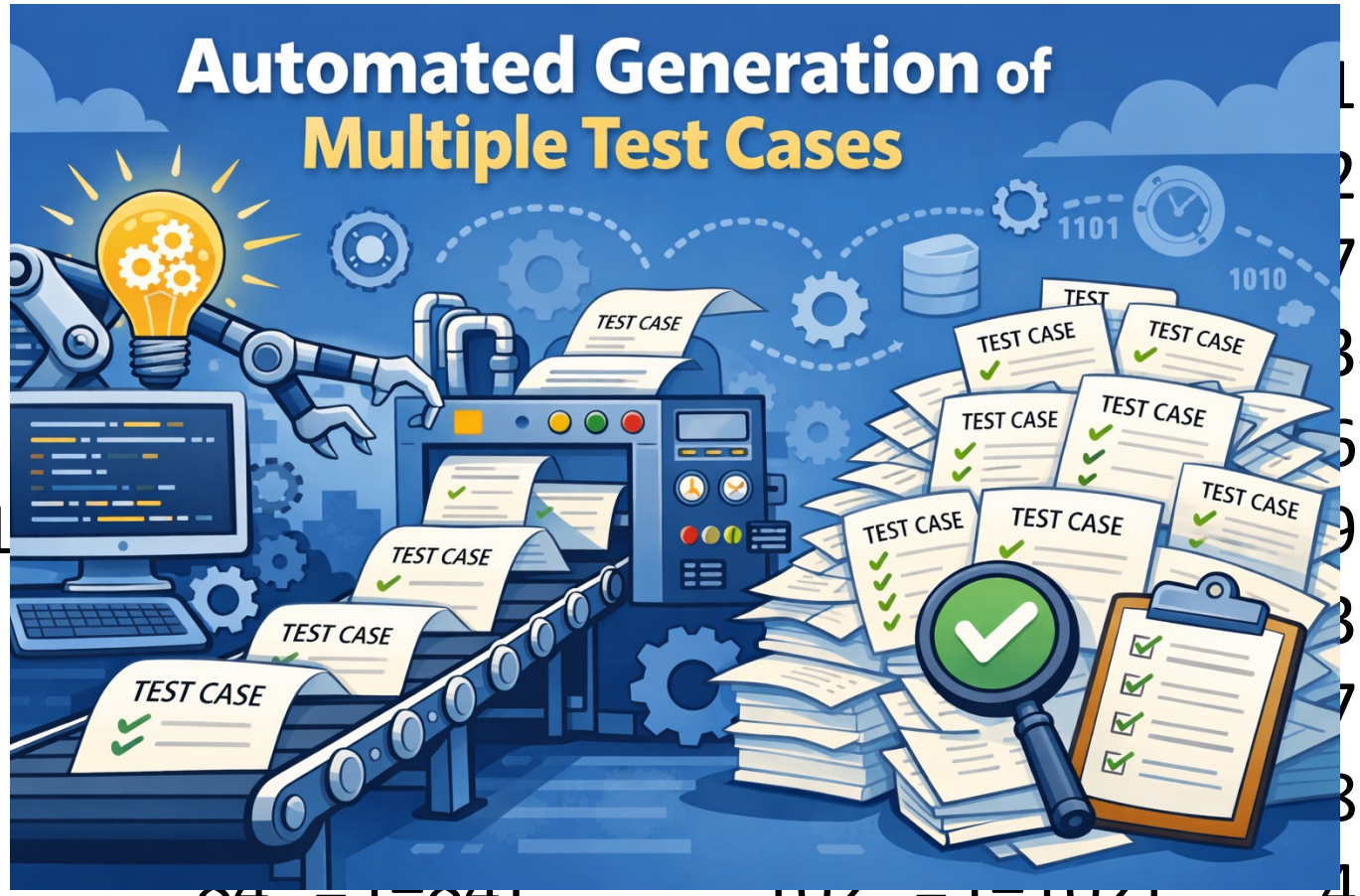
bug found

MT Enables Automated Test Generation (at scale)

One MR = one oracle for many automatically generated tests

$x_1 = -x_2 \rightarrow square(x_1) = square(x_2)$

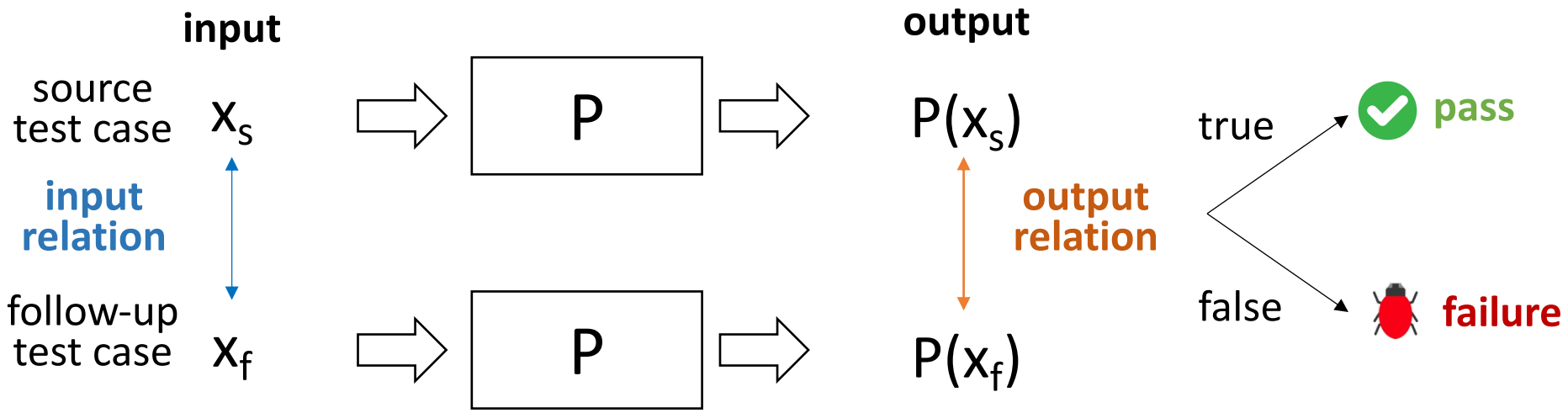
$12^2 = (-12)^2$
$7^2 = (-7)^2$
$25^2 = (-25)^2$
$3^2 = (-3)^2$
$48^2 = (-48)^2$
$101^2 = (-101$
$6^2 = (-6)^2$
$19^2 = (-19)^2$
$88^2 = (-88)^2$
$14^2 = (-14)^2$



$112^2 = (-112)^2$
$23^2 = (-23)^2$
$75^2 = (-75)^2$
$34^2 = (-34)^2$
$60^2 = (-60)^2$
$98^2 = (-98)^2$
$37^2 = (-37)^2$
$71^2 = (-71)^2$
$83^2 = (-83)^2$
$40^2 = (-40)^2$

MT Enables Automated Test Generation (at scale)

One MR = one oracle for many automatically generated tests



Workflow:

Step 1) auto-generate a **source** test
Step 2) apply the MR input transformation to create a **follow-up** test
Step 3) run system on both inputs
Step 4) raise **test failure** if the output relation is violated

Metamorphic relation (Test Oracle)

input relation → output relation

Traditional Testing (input + expected output)

natural language inference task, a core NLP task that determines the relationship between a **premise** and a **hypothesis**

input:

PR: A man is playing the flute into the microphone.
HY: A man is playing at a concert.

expected result: **neutral**

costly manual label

```
Premise: \{PR}\
Hypothesis: \{HY}\

Is this entailment, contradiction,
or neutral? Only write a one-word
answer.
```

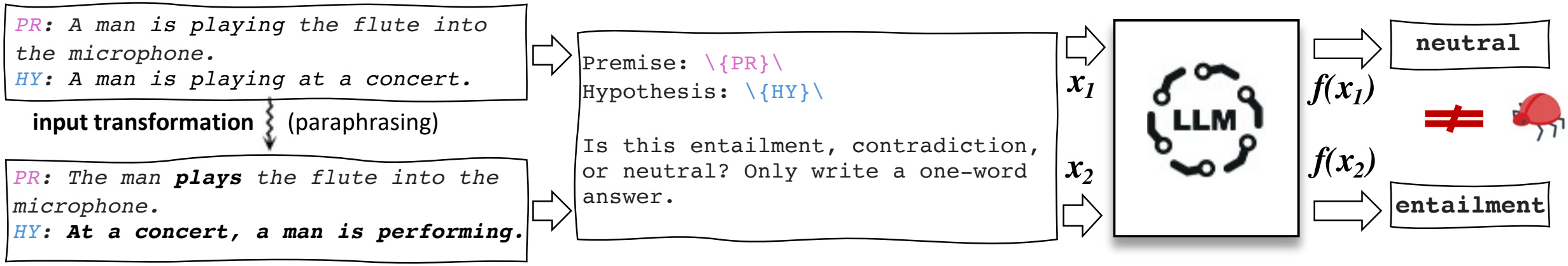
LLM

neutral = **neutral**



Metamorphic Testing

Metamorphic Relation (MR): x_2 is a paraphrase of $x_1 \rightarrow f(x_1) = f(x_2)$



real bug that we found in GPT4

Project 36: A Framework for Automating Metamorphic Testing Using LLMs

Soham Kulkarni - Nathan Turley

Problem: Metamorphic Testing (MT) is powerful but hard and costly to set up, so it is mostly used by large tech companies

Goal: Design an automated framework that makes MT practical for everyday developers

Approach: Simplify how metamorphic relations (MRs) are defined, and automate test generation and execution

AI support: Use large language models (LLMs) to assist with automated test generation behind the scenes

Key output: Develop a novel **JUnit extension** for writing MT-based test cases with minimal developer effort

Evaluation: Assess both
- **Usability**: how easy the framework is to learn and use (ethics application needed)
- **Effectiveness**: how well it improves fault detection versus conventional tests

Project 36: A Framework for Automating Metamorphic Testing Using LLMs

Soham Kulkarni - Nathan Turley

Start by reading this:



RESEARCH-ARTICLE | FREE ACCESS |

MTF: an Open-Source Metamorphic Testing Framework for LLM-based systems

Authors: Theis Henry, Sian Savourat, Lydie du Bousquet, Masahide Nakamura | Authors Info & Claims

AIAT '25: Proceedings of the 2025 5th International Conference on Artificial Intelligence and Application Technologies
Pages 62 - 66 • https://doi.org/10.1145/3787120.3787123

Published: 08 February 2026 Publication History | Check for updates

0 108 | PDF/eReader

https://dl.acm.org/doi/10.1145/3787120.3787123

Project 42: Metamorphic Testing of LLMs via Prompt Transformations

Liam Byrne - Manan Patel

Background: Our recent work has shown that Metamorphic Testing (MT) can effectively test LLMs by defining metamorphic relations (MRs) and automatically generating follow-up tests

Project focus: Investigate MRs for coding tasks performed by LLMs (Code generation, code completion, bug fixing, refactoring, and code explanation)

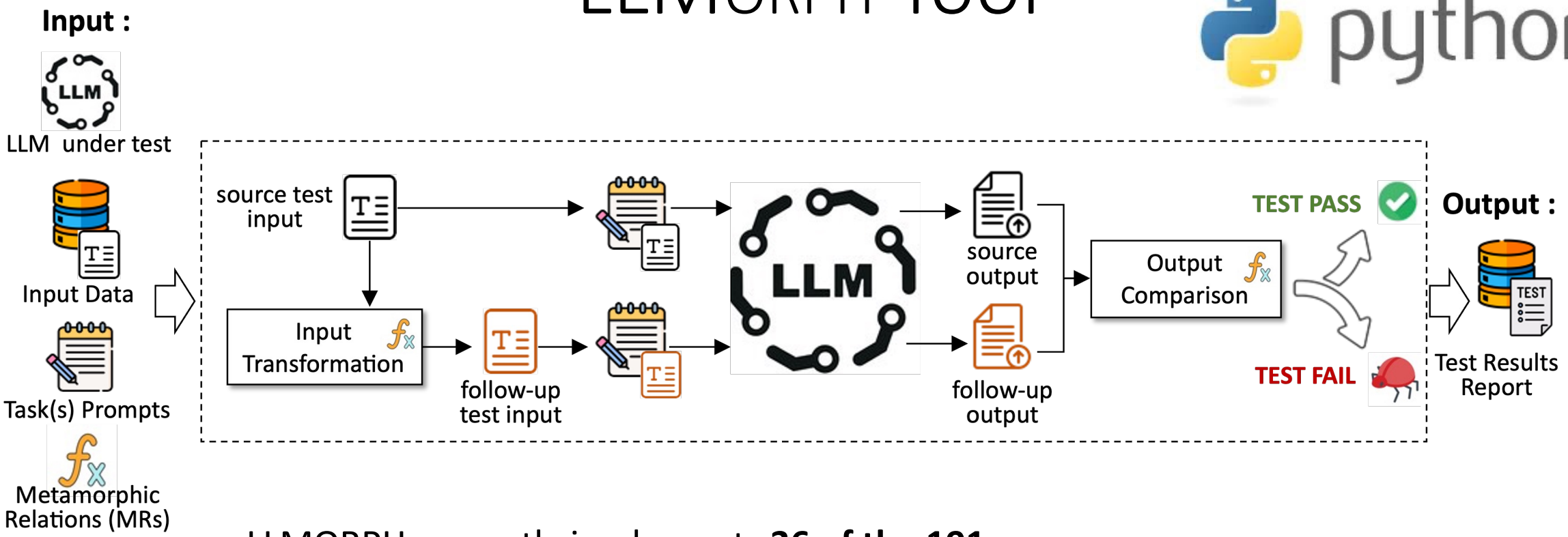
Main goal: Identify and define coding-related MRs, where transformed prompts or code-related inputs should lead to outputs that are

- Consistent
- Functionally equivalent
- Predictably different

Implementation: Extend our Python-based MT tool by adding Coding-task-specific MR operators

Success metrics: Compare fault detection rates and false positives

LLMorph Tool



LLMORPH currently implements **36 of the 191** covering **4 NLP tasks**.

Input transformations: implemented with LLM if complex transformation is required

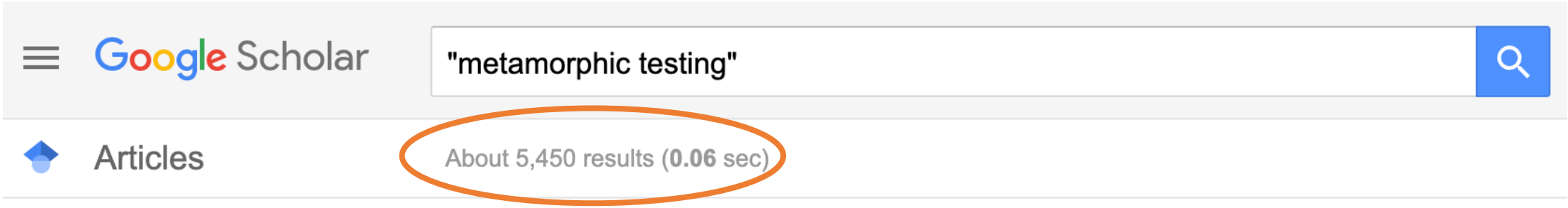
Output relation: Syntactic or Semantic (BERT)

Project 43: Understanding the Adoption of Metamorphic Testing in Software Development

Ming Ray Goy - Ethan Quach

Metamorphic Testing (adoption)

Popular Testing Technique



Used by:

Meta Google NASA GCC YouTube Spotify .....

What about non-big-tech companies ?

Project 43: Understanding the Adoption of Metamorphic Testing in Software Development

Ming Ray Goy - Ethan Quach

Problem: Despite its potential, Metamorphic Testing (MT) is still rarely used outside major tech companies

Goal: Investigate why MT has not been widely adopted in everyday software development

Developer study: Use surveys and interviews to explore (ETHICS APPLICATION)

- Developers' awareness of MT
- When they see MT as useful
- Obstacles to adopting MT in practice

Open-source analysis: Mine GitHub repositories to look for evidence of MT usage in real projects

Implementation work: Develop scripts to download and analyse open-source repositories at scale

Expected outcome: Identify practical barriers and opportunities for broader MT adoption in the developer community